Secrets of Style

Armario digital con inteligencia artificial

Base de datos, gestión de la información y arquitectura de software

Fundamentos de Inteligencia Artificial
Universidad Tecnológica del Valle de Toluca
Equipo Secrets of Style
6 de noviembre de 2026

Índice

1. El proyecto 3
1.0 Nombre del proyecto 3
1.1 Descripción del proyecto 3
1.2 Planteamiento del problema 3
1.3 Modelo relacional 4
1.4 Diccionario de datos 6
2. Gestión de la información 12
2.0 Definición de las necesidades de información 12
2.1 Descripción de la ubicación y características de los datos 12
2.2 Descripción de las tareas y sus técnicas de automatización 14
2.3 Descripción del método de importación y exportación 15
2.4 Lineamientos de seguridad de la base de datos 16
3. Arquitectura de software 18
3.0 Requerimientos funcionales y no funcionales 18
3.1 Justificación de la arquitectura seleccionada 20
3.2 Diagrama de arquitectura de software 20
3.3 Características y funciones de los componentes 21
3.4 Diagramas de la arquitectura 22

1. El proyecto

1.0 Nombre del proyecto

Secrets of Style: armario digital con inteligencia artificial que arma atuendos con la ropa de la persona, según el clima real, la ocasión y sus gustos.

1.1 Descripción del proyecto

Secrets of Style es una aplicación móvil, hecha con React Native y Expo, que trabaja con un servidor propio en Python (FastAPI y SQLite). La persona fotografía cada prenda y un modelo de visión que corre en la misma computadora del servidor (Ollama con `gemma3:4b`) propone su categoría, color, estampado, estilo y nivel de abrigo. La persona revisa esos datos antes de guardar.

Para vestirse, la persona elige una ocasión (día a día, formal o deporte) y una prenda base. El servidor consulta el clima real de su ubicación en Open-Meteo y arma los atuendos en tres etapas:

1. **Lógica proposicional.** Con 19 proposiciones y 18 reglas, el encadenamiento hacia adelante deduce qué prendas son aptas para el clima y la ocasión, y qué colores y estampados combinan.
2. **Búsqueda.** A* con una heurística admisible encuentra los 12 atuendos de menor costo. El costo suma color, clima y penalizaciones de la lógica; una prenda fuera de la ocasión cuesta +20, más que todo lo demás junto.
3. **Perceptrón.** Escrito en Python puro, reordena esos candidatos según los gustos de la persona, que aprende de los corazones que da y de un cuestionario inicial. Se muestran los tres mejores.

Cada atuendo se puede explicar en lenguaje natural: el modelo local redacta solo con los hechos que dedujo la lógica. La app también guarda atuendos con el clima del día y muestra estadísticas del armario. Está en desarrollo un **probador virtual**: el modelo CatVTON viste la foto de la persona con las prendas del look. El servidor y el motor del probador ya funcionan y están probados; falta la pantalla de la app.

1.2 Planteamiento del problema

Decidir qué ponerse es una tarea diaria que consume tiempo y suele resolverse con las mismas pocas prendas. Una parte del armario se olvida y la combinación no siempre toma en cuenta el clima real del día ni la ocasión. Las aplicaciones de moda disponibles suelen recomendar ropa de un catálogo para venderla, no la que la persona ya tiene. Además, casi ninguna explica sus sugerencias y la mayoría envía las fotos a servicios en la nube.

Pregunta de investigación: ¿cómo recomendar atuendos completos con la ropa que la persona ya tiene, adecuados al clima real y a la ocasión, que aprendan de sus gustos y expliquen por qué funcionan, sin enviar sus fotos a terceros y con el equipo de cómputo del equipo (una laptop con una GPU de 6 GB)?

Objetivo general: desarrollar un sistema cliente-servidor que registre el armario de la persona y genere atuendos explicables mediante lógica proposicional, búsqueda heurística y aprendizaje de preferencias, con modelos de IA que corran en la misma computadora.

Objetivos específicos:

- Registrar prendas con foto y datos, y proponer esos datos con un modelo de visión local.
- Representar el conocimiento sobre clima, color, estampado, estilo y ocasión con reglas de lógica proposicional.
- Encontrar los atuendos de menor costo con A* y comparar el resultado con BFS, DFS, UCS y Greedy en el servidor.
- Aprender los gustos de cada persona con un perceptrón entrenado con corazones y un cuestionario inicial.
- Guardar la información en una base de datos relacional con integridad referencial y aislamiento por cuenta.
- Mostrar el armario, los atuendos y las estadísticas en una app móvil accesible, en modo claro y oscuro.

Alcance: el sistema corre en una red local. Cada cuenta ve solo su propio armario. No incluye compras, redes sociales ni recuperación de contraseña por correo.

1.3 Modelo relacional

La base de datos es un solo archivo SQLite (`backend/secrets_of_style.db`) con 10 tablas. La tabla `usuarios` es el centro: todas las demás, salvo `climas`, dependen de una cuenta y se borran en cascada si la cuenta se elimina. La Figura 1 muestra las tablas, sus llaves y la cardinalidad de cada relación.

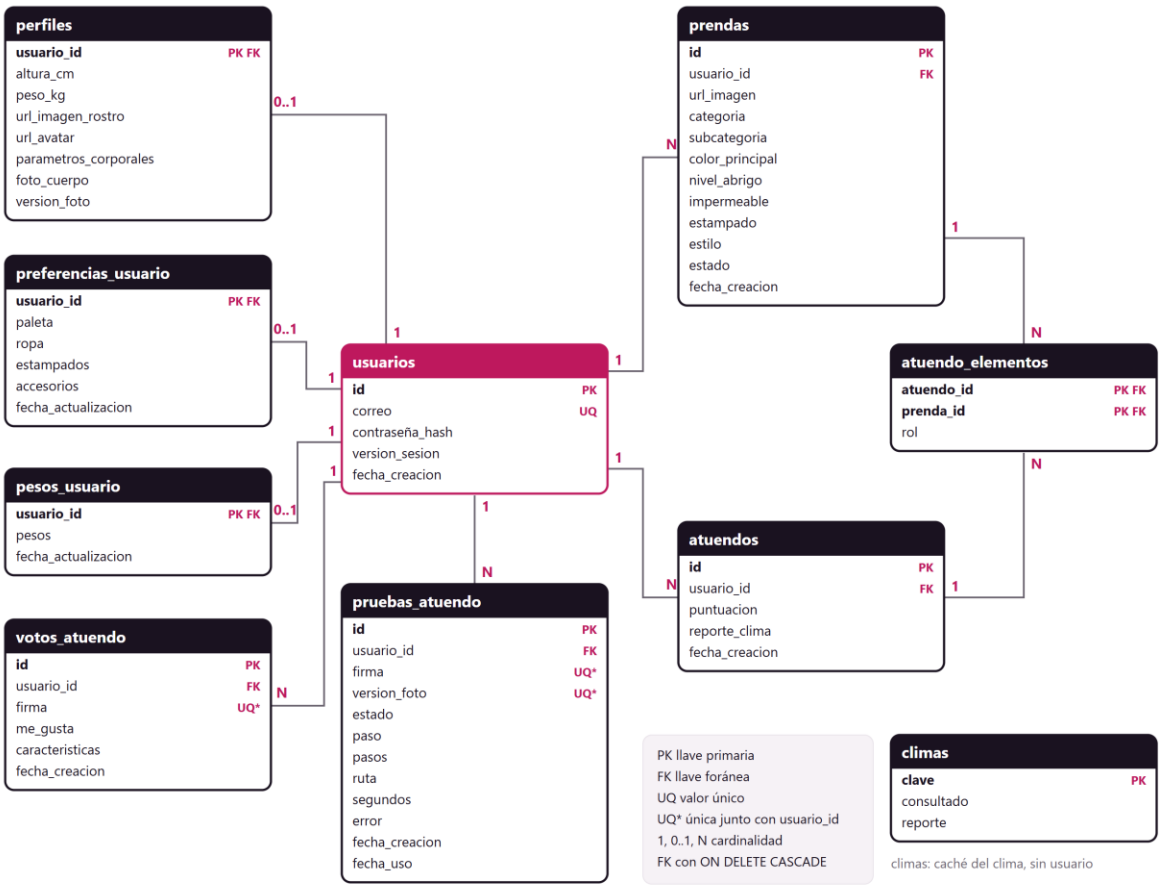


Figura 1. Modelo relacional de la base de datos secrets_of_style.db.

Tabla 1. Relaciones entre tablas

Tabla padre	Tabla hija	Cardinalidad	Llave foránea	Al borrar el padre
usuarios	perfiles	1 : 0..1	perfiles.usuario_id (también es la PK)	CASCADE
usuarios	preferencias_usuario	1 : 0..1	preferencias_usuario.usuario_id (PK)	CASCADE
usuarios	pesos_usuario	1 : 0..1	pesos_usuario.usuario_id (PK)	CASCADE
usuarios	prendas	1 : N	prendas.usuario_id	CASCADE
usuarios	atuendos	1 : N	atuendos.usuario_id	CASCADE
usuarios	votos_atuendo	1 : N	votos_atuendo.usuario_id	CASCADE
usuarios	pruebas_atuendo	1 : N	pruebas_atuendo.usuario_id	CASCADE
atuendos	atuendo_elementos	1 : N	atuendo_elementos.atuendo_id	CASCADE
prendas	atuendo_elementos	1 : N	atuendo_elementos.prenda_id	CASCADE

Relación varios a varios. Un atuendo tiene varias prendas y una prenda puede estar en varios atuendos. La tabla `atuendo_elementos` resuelve esa relación con una llave primaria compuesta (`atuendo_id`, `prenda_id`) y guarda el rol de cada prenda en el atuendo.

Restricciones de unicidad. `usuarios.correo` es único. En `votos_atuendo`, la pareja (`usuario_id`, `firma`) es única: una persona tiene un solo voto por combinación de prendas y, si cambia de opinión, el voto se reemplaza. En `pruebas_atuendo`, la terna (`usuario_id`, `firma`, `version_foto`) es única, así que el mismo look con la misma foto no se genera dos veces.

Normalización. Las tablas cumplen la tercera forma normal en sus datos de negocio. Hay cuatro columnas JSON a propósito: `atuendos.reporte_clima` y `votos_atuendo.caracteristicas` son fotografías del momento (el clima del día en que se guardó y las características del look cuando se votó), que no deben cambiar si después se edita o se borra una prenda. `pesos_usuario.pesos` y `perfiles.parametros_corporales` tienen una estructura que puede crecer sin cambiar el esquema.

1.4 Diccionario de datos

Los tipos son los declarados en la base real. Las primeras tablas se crearon con tipos como `VARCHAR`, `FLOAT`, `DATETIME` o `JSON`. SQLite los guarda con la afinidad equivalente (texto, real o entero), y una instalación nueva crea las mismas columnas con `TEXT`, `REAL` e `INTEGER`. En la columna Nulo, «No» significa que el valor es obligatorio (`NOT NULL`).

usuarios

Cuentas de acceso. Hoy tiene 4 filas.

Tabla 2. Diccionario de la tabla usuarios

Campo	Tipo	Nulo	Llave	Descripción	Valores o ejemplo
id	INTEGER	No	PK	Identificador de la cuenta, autoincremental.	1
correo	VARCHAR(255)	No	UQ	Correo de acceso, guardado en minúsculas y sin espacios.	persona@correo.com
contraseña_hash	VARCHAR(255)	No		Sal y resumen PBKDF2-HMAC-SHA256 con 100 000 iteraciones, en hexadecimal. Nunca se guarda la contraseña.	sal$resumen (32 y 64 caracteres)
fecha_creacion	DATETIME	No		Fecha y hora de registro (UTC).	2026-10-13 18:02:11
version_sesion	INTEGER	Sí		Sube en 1 al cerrar sesión; los tokens con una versión anterior dejan de servir.	0, 1, 2…

perfiles

Datos corporales y la foto del probador. Una fila por cuenta (4 filas).

Tabla 3. Diccionario de la tabla perfiles

Campo	Tipo	Nulo	Llave	Descripción	Valores o ejemplo
usuario_id	INTEGER	No	PK, FK	Cuenta dueña del perfil (relación 1 a 1).	1
altura_cm	FLOAT	Sí		Altura en centímetros. Solo por la API; la app aún no la pide.	170
peso_kg	FLOAT	Sí		Peso en kilogramos. Solo por la API.	65
url_imagen_rostro	VARCHAR(500)	Sí		Ruta pública de una foto de rostro. Reservada, sin uso en la app.	/archivos/rostros/<uuid>.jpg
url_avatar	VARCHAR(500)	Sí		Reservada para un avatar. Sin uso.	NULL
parametros_corporales	JSON	Sí		IMC, complexión y escala de altura calculados a partir de altura y peso.	{"imc": 22.5, "complexion": "media", "escala_altura": 1.0}
foto_cuerpo	TEXT	Sí		Ruta privada de la foto de cuerpo completo del probador. Nunca se envía a la app.	backend\privado\cuerpos\1_<uuid>.jpg
version_foto	INTEGER	Sí		Sube al cambiar o borrar la foto; invalida las imágenes generadas con la anterior.	0, 1, 2…

prendas

El armario. Hoy hay 35 prendas: 12 partes superiores, 9 calzados, 6 partes inferiores, 6 abrigos y 2 accesorios.

Tabla 4. Diccionario de la tabla prendas

Campo	Tipo	Nulo	Llave	Descripción	Valores o ejemplo
id	INTEGER	No	PK	Identificador de la prenda.	28
usuario_id	INTEGER	No	FK	Cuenta dueña de la prenda.	1
url_imagen	VARCHAR(500)	No		Ruta pública de la foto, con nombre UUID.	/archivos/prendas/<uuid>.jpg
categoria	VARCHAR(30)	No		Tipo de prenda.	parte_superior, parte_inferior, calzado, abrigo, accesorio
subcategoria	VARCHAR(60)	Sí		Nombre libre de hasta 60 caracteres.	Camisa de vestir, Jeans

Campo	Tipo	Nulo	Llave	Descripción	Valores o ejemplo
color_principal	VARCHAR(30)	No		Color que cubre más área. 7 neutros y 7 vivos.	Neutros: negro, blanco, gris, beige, cafe, azul_marino, mezclilla. Vivos: rojo, azul, verde, amarillo, rosa, morado, naranja
nivel_abrigo	INTEGER	No		Qué tanto abriga, de 1 (muy fresca) a 5 (muy abrigada).	1 a 5
impermeable	BOOLEAN	No		Si resiste la lluvia.	0 o 1
estado	VARCHAR(20)	No		Disponibilidad. Solo las disponibles entran en los atuendos.	disponible, en_lavado, archivada
fecha_creacion	DATETIME	No		Fecha en que se registró. Alimenta la gráfica de historial.	2026-10-30 12:41:07
estampado	TEXT	Sí		Estampado; dos estampados juntos cuestan +2 en A*. Por defecto liso.	liso, rayas, cuadros, floral, animal, otro
estilo	TEXT	Sí		Estilo que se cruza con la ocasión. Por defecto casual.	casual, semiformal, formal, deportivo, urbano

atuendos

Atuendos guardados por la persona (6 filas).

Tabla 5. Diccionario de la tabla atuendos

Campo	Tipo	Nulo	Llave	Descripción	Valores o ejemplo
id	INTEGER	No	PK	Identificador del atuendo.	4
usuario_id	INTEGER	No	FK	Cuenta dueña.	1
puntuacion	FLOAT	No		Costo que le dio A*; menor es mejor.	3.5
reporte_clima	JSON	Sí		Clima del día en que se guardó, como fotografía del momento.	{"temperatura_c": 14.2, "probabilidad_lluvia": 40, …}
fecha_creacion	DATETIME	No		Cuándo se guardó. Ordena la línea de tiempo de Guardados.	2026-10-30 08:15:00

atuendo_elementos

Prendas de cada atuendo guardado (20 filas). Resuelve la relación varios a varios.

Tabla 6. Diccionario de la tabla atuendo_elementos

Campo	Tipo	Nulo	Llave	Descripción	Valores o ejemplo
atuendo_id	INTEGER	No	PK, FK	Atuendo al que pertenece.	4
prenda_id	INTEGER	No	PK, FK	Prenda incluida. Si se borra la prenda, se quita de los atuendos.	28

Campo	Tipo	Nulo	Llave	Descripción	Valores o ejemplo
rol	VARCHAR(30)	No		Lugar de la prenda en el atuendo (su categoría).	parte_superior, parte_inferior, calzado, abrigo, accesorio

climas

Caché del clima por coordenadas (5 filas). No depende de una cuenta.

Tabla 7. Diccionario de la tabla climas

Campo	Tipo	Nulo	Llave	Descripción	Valores o ejemplo
clave	TEXT	Sí	PK	Latitud y longitud redondeadas a 2 decimales (unos 1.1 km).	19.28,-99.65
consultado	REAL	No		Momento de la consulta, en segundos desde 1970. Vale 30 minutos.	1793469600.5
reporte	TEXT	No		JSON con temperatura, sensación térmica, lluvia, descripción, hechos lógicos y deducciones.	{"temperatura_c": 13.1, "hechos": ["frio"], …}

preferencias_usuario

Respuestas del cuestionario de estilo (2 filas). Dan los pesos iniciales del perceptrón.

Tabla 8. Diccionario de la tabla preferencias_usuario

Campo	Tipo	Nulo	Llave	Descripción	Valores o ejemplo
usuario_id	INTEGER	Sí	PK, FK	Cuenta que contestó.	1
paleta	TEXT	No		Colores que usa más.	sin_preferencia, neutros, vivos, oscuros, claros
ropa	TEXT	No		Lo que se pone más seguido.	sin_preferencia, casual, urbano, arreglado, deportiva
estampados	TEXT	No		Relación con los estampados.	encantan, uno, liso
accesorios	TEXT	No		Qué tan seguido usa accesorios. «casi_nunca» quita el accesorio del atuendo.	casi_siempre, a_veces, casi_nunca
fecha_actualizacion	TEXT	Sí		Última vez que se contestó.	CURRENT_TIMESTAMP

pesos_usuario

Pesos del perceptrón de cada persona (2 filas). Se recalculan con cada voto.

Tabla 9. Diccionario de la tabla pesos_usuario

Campo	Tipo	Nulo	Llave	Descripción	Valores o ejemplo
usuario_id	INTEGER	Sí	PK, FK	Cuenta dueña.	1

Campo	Tipo	Nulo	Llave	Descripción	Valores o ejemplo
pesos	TEXT	No		JSON {característica: peso}, con el sesgo. Las características nuevas empiezan en 0.	{"tono": -0.25, "estilo_urbano": 0.5, "sesgo": 0.25, …}
fecha_actualizacion	TEXT	Sí		Último reentrenamiento.	CURRENT_TIMESTAMP

votos_atuendo

Ejemplos de entrenamiento del perceptrón (41 filas).

Tabla 10. Diccionario de la tabla votos_atuendo

Campo	Tipo	Nulo	Llave	Descripción	Valores o ejemplo
id	INTEGER	Sí	PK	Identificador del voto.	17
usuario_id	INTEGER	No	FK, UQ*	Cuenta que votó.	1
firma	TEXT	No	UQ*	Ids de las prendas del look, ordenados y unidos con guion.	21-35-37-39
me_gusta	INTEGER	No		1 con corazón o guardado; -1 si se mostró y no recibió corazón.	1 o -1
caracteristicas	TEXT	No		JSON con las 13 características del look al votar, de 0 a 1. Se calculan en el servidor.	neutros, contraste, tono, complementarios, monocromo, abrigo, estampado, accesorio y 5 de estilo
fecha_creacion	TEXT	Sí		Cuándo se votó.	CURRENT_TIMESTAMP

pruebas_atuendo

Cola y caché del probador virtual. Hoy está vacía porque la pantalla de la app aún no existe.

Tabla 11. Diccionario de la tabla pruebas_atuendo

Campo	Tipo	Nulo	Llave	Descripción	Valores o ejemplo
id	INTEGER	Sí	PK	Identificador de la prueba.	1
usuario_id	INTEGER	No	FK, UQ*	Cuenta dueña.	1
firma	TEXT	No	UQ*	Prendas que se visten (sin calzado ni accesorios).	2-28
version_foto	INTEGER	No	UQ*	Versión de la foto con la que se generó.	1
estado	TEXT	No		Avance del trabajo.	en_cola, procesando, lista, error
paso	INTEGER	Sí		Pasadas terminadas.	0, 1, 2

Campo	Tipo	Nulo	Llave	Descripción	Valores o ejemplo
pasos	INTEGER	No		Pasadas necesarias: inferior y luego superior o abrigo.	1 o 2
ruta	TEXT	Sí		Ruta privada de la imagen generada.	backend\privado\pruebas\1\<uuid>.jpg
segundos	REAL	Sí		Tiempo que tardó.	64.3
error	TEXT	Sí		Mensaje si falló.	El probador está apagado…
fecha_creacion	TEXT	Sí		Cuándo se pidió.	CURRENT_TIMESTAMP
fecha_uso	TEXT	Sí		Último uso. Al pasar de 40 pruebas se borran las de uso más viejo.	CURRENT_TIMESTAMP

2. Gestión de la información

Esta sección describe los datos reales del proyecto al 6 de noviembre de 2026: qué información se necesita, dónde está, qué tareas la procesan, cómo entra y sale del sistema y cómo se protege.

2.0 Definición de las necesidades de información

Cada necesidad nace de una pregunta que se hacen la persona usuaria o el propio sistema. La Tabla 12 relaciona cada pregunta con los datos que la responden.

Tabla 12. Necesidades de información

Pregunta	Quién la necesita	Datos que la responden	Origen
¿Quién es la persona y su sesión sigue vigente?	Seguridad (todas las rutas protegidas)	Correo, resumen de la contraseña, versión de sesión, firma y vencimiento del token	usuarios; .clave_secreta
¿Qué ropa tengo y cuál está disponible?	Persona (Armario)	Foto, categoría, color, estampado, estilo, abrigo, impermeabilidad y estado	prendas; archivos_subidos
¿Qué datos tiene esta prenda nueva?	Persona al registrar	Propuesta de categoría, color, estampado, estilo y abrigo	Ollama (gemma3:4b), sin guardarse hasta que la persona confirme
¿Qué clima hace donde estoy?	Lógica y A*	Temperatura, sensación térmica, probabilidad de lluvia y hechos (frío, lluvia…)	Open-Meteo; caché en climas
¿Qué me pongo hoy para esta ocasión?	Persona (Atuendos)	Prendas disponibles, hechos del clima y de la ocasión, costos de A*	prendas; climas; reglas de logica.py
¿Qué looks me gustan?	Perceptrón	Votos con sus 13 características, pesos y respuestas del cuestionario	votos_atuendo; pesos_usuario; preferencias_usuario
¿Por qué funciona este atuendo?	Persona	Hechos deducidos por la lógica, redactados por el modelo local	logica.py; Ollama
¿Qué atuendos guardé y con qué clima?	Persona (Guardados)	Atuendo, prendas, rol, costo y clima del día	atuendos; atuendo_elementos
¿Cómo está compuesto mi armario?	Persona (Estadísticas)	Conteos por categoría, color, abrigo e impermeabilidad; historial por fecha	prendas; atuendos
¿Cómo me veo con este look? (en desarrollo)	Persona (probador)	Foto de cuerpo completo, prendas del look e imagen generada	perfiles.foto_cuerpo; pruebas_atuendo; privado

2.1 Descripción de la ubicación y características de los datos

Todos los datos viven en la computadora que corre el servidor, dentro de la carpeta `backend`. El teléfono solo guarda el token de sesión y una caché de imágenes. La información es de tres tipos: **estructurada** (tablas), **semiestructurada** (JSON dentro de algunas columnas) y **no estructurada** (fotos).

Tabla 13. Ubicación, formato y volumen real de los datos

Conjunto de datos	Ubicación	Formato	Volumen al 6/11/2026	Características
Base de datos	backend\secrets_of_style.db	SQLite 3, un archivo	108 KB; 10 tablas; 120 filas en total	Estructurada. Llaves foráneas activas en cada conexión.
Fotos de prendas	backend\archivos_subidos\prendas\	JPEG y WEBP, ~450 px	55 archivos, 3.3 MB; promedio de 86 KB	No estructurada. Nombres UUID. Se sirven en /archivos.
Fotos del probador	backend\privado\	JPEG	0 archivos (la pantalla aún no existe)	Datos personales sensibles. Solo se entregan con sesión.
Clave de firma	backend\.clave_secreta	Texto, 64 caracteres hexadecimales	1 archivo	Secreto. Se crea sola la primera vez. No va al repositorio.
Caché del clima	Tabla climas	JSON por coordenadas	5 ubicaciones (zona de Toluca y CDMX)	Vence a los 30 minutos.
Modelos de IA	Ollama y caché de Hugging Face del usuario de Windows	Pesos de redes neuronales	gemma3:4b 3.3 GB; probador ~4 GB	Se descargan una vez. No contienen datos de personas.
Sesión en el teléfono	SecureStore (llavero del sistema)	Texto	1 token por teléfono	Cifrado por el sistema operativo.

Tabla 14. Filas por tabla

Tabla	Filas	Tabla	Filas
usuarios	4	atuendo_elementos	20
perfiles	4	climas	5
prendas	35	pesos_usuario	2
atuendos	6	votos_atuendo	41
preferencias_usuario	2	pruebas_atuendo	0

Crecimiento esperado. Cada prenda agrega una fila de unos 300 bytes y una foto de unos 86 KB; un armario de 100 prendas ocuparía cerca de 9 MB en fotos. Cada vez que se generan atuendos se agregan hasta 3 filas en `votos_atuendo`. Cada imagen del probador pesa unos 300 KB y hay un máximo de 40 por persona, unos 12 MB. Por este volumen, SQLite en un solo archivo es suficiente.

Hallazgo de la revisión. En `archivos_subidos\prendas` hay 55 fotos, pero solo 35 están referenciadas por una prenda. Las otras 20 (191 KB) son de prendas borradas en versiones anteriores, antes de que borrar una prenda también borrara su foto. Se recomienda una tarea de limpieza (sección 2.2).

2.2 Descripción de las tareas y sus técnicas de automatización

La Tabla 15 resume las tareas que el sistema hace sin que nadie las ejecute a mano. Todas están en el código del servidor y se comprueban con pruebas automáticas.

Tabla 15. Tareas automatizadas

Tarea	Cuándo ocurre	Técnica	Dónde
Crear y migrar el esquema	Al arrancar el servidor	`CREATE TABLE IF NOT EXISTS`; las columnas nuevas se detectan con `PRAGMA table_info` y se agregan con `ALTER TABLE`	base_datos.crear_tablas
Mantener la integridad referencial	En cada conexión	`PRAGMA foreign_keys = ON` y `ON DELETE CASCADE`	base_datos.conectar
Guardar en caché el clima	En cada recomendación	Consulta por coordenadas redondeadas; vigencia de 30 min; `INSERT OR REPLACE`; si no hay red, usa el último guardado y lo avisa	clima.obtener_clima
Proponer los datos de una prenda	Al tomar la foto	Modelo de visión con salida estructurada (JSON schema) y limpieza contra los catálogos	vision.detectar_prenda
Deducir hechos	En cada recomendación	Encadenamiento hacia adelante sobre 18 reglas	logica.py
Armar atuendos	En cada recomendación	A* k-mejores con heurística admisible	busqueda.py
Registrar "no me gusta" implícito	Al mostrar los looks	`INSERT … ON CONFLICT DO NOTHING` con -1 para cada look sin voto	principal.registrar_sin_corazon
Reentrenar el perceptrón	Con cada voto, al guardar un atuendo o al contestar el cuestionario	Regla del perceptrón, 10 épocas sobre todos los votos, desde los pesos del cuestionario; upsert de los pesos	principal.reentrenar; aprendizaje.entrenar
Cambiar de opinión sin duplicar	Al votar	`ON CONFLICT (usuario_id, firma) DO UPDATE`	principal.guardar_voto
Atender la cola del probador	En segundo plano	Un hilo toma la prueba más antigua; al arrancar retoma las que quedaron a medias	probador.trabajar
Reutilizar resultados del probador	Al pedir un look	Caché por firma de prendas y versión de foto; reutiliza la pasada del pantalón entre looks	probador.pedir; probador.procesar
Repartir la GPU	Antes de usar Gemma o CatVTON	Un candado (`threading.Lock`): quien usa la GPU saca al otro modelo	gpu.py; ollama.py
Borrar archivos ligados	Al borrar una prenda o cambiar la foto	Se borran la foto, las pruebas que la usan y sus archivos	principal.eliminar_prenda; probador.olvidar_prenda
Limitar el espacio del probador	Al terminar cada prueba	Conserva las 40 de uso más reciente por persona	probador.limpiar_viejas
Comprobar el sistema	Antes de cada entrega	6 scripts con 244 comprobaciones sobre bases y carpetas temporales	backend\pruebas

Tareas que hoy son manuales y conviene automatizar:

- **Respaldo de la base y las fotos.** Hoy se copia a mano (sección 2.3). Se puede programar con el Programador de tareas de Windows.
- **Limpieza de fotos huérfanas.** Un script que compare los archivos de `archivos_subidos\prendas` con `prendas.url_imagen` y borre los que nadie usa.
- **Borrar los climas viejos.** La tabla `climas` crece con cada ubicación nueva; las filas de más de un día se pueden borrar sin perder nada.

2.3 Descripción del método de importación y exportación

El sistema intercambia datos en formatos abiertos: JSON para los datos, multipart/form-data para subir fotos y JPEG, PNG o WEBP para las imágenes. La Tabla 16 separa lo que ya está implementado de los procedimientos que se hacen a mano.

Tabla 16. Métodos de importación y exportación

Flujo	Dirección	Formato	Método	Estado
Fotos de prendas	Importación	multipart/form-data → JPEG/WEBP	La app recorta a 3:4 y reduce a 1200 px en JPEG; el servidor valida la firma de los bytes y el tamaño (8 MB) y guarda con nombre UUID. `POST /api/prendas`	Implementado
Clima	Importación	JSON por HTTPS	Petición a Open-Meteo; se convierte a hechos lógicos y se guarda en `climas`	Implementado
Datos propuestos por la IA	Importación	JSON con esquema	Ollama devuelve solo valores de los catálogos; la persona confirma	Implementado
Prendas de ejemplo	Importación	Fotos + datos	17 prendas con fotos de dominio público (Wikimedia Commons) cargadas por la misma API	Hecho una vez
Foto del probador	Importación	multipart/form-data	El motor revisa que se vean rostro, torso y piernas antes de aceptarla. `POST /api/perfil/foto`	Servidor listo
Datos de la persona	Exportación	JSON	`GET /api/prendas`, `/api/atuendos`, `/api/estadisticas` y `/api/perfil`, con su token	Implementado
Imágenes	Exportación	JPEG/WEBP	`/archivos/...` para prendas; `/api/atuendos/probador/{id}/imagen` con token para el probador	Implementado
Respaldo completo	Exportación	Archivo SQLite + carpetas	Copia en caliente con la API de respaldo de sqlite3 (abajo) y copia de `archivos_subidos` y `.clave_secreta`	Manual
Tabla a CSV	Exportación	CSV UTF-8	Script corto con los módulos `sqlite3` y `csv` (abajo)	Manual
Restauración	Importación	Archivo SQLite	Detener el servidor, reemplazar `secrets_of_style.db` y las carpetas, y arrancarlo; las migraciones agregan lo que falte	Manual

Respaldo en caliente. La API de respaldo de sqlite3 copia la base aunque el servidor esté corriendo, sin dejarla a medias. Desde la carpeta `backend`:

```
.venv\Scripts\python -c "import sqlite3;
sqlite3.connect('secrets_of_style.db').backup(sqlite3.connect('respaldo.db'))"
```

Exportar una tabla a CSV (por ejemplo, `prendas`):

```
.venv\Scripts\python -c "import csv,sqlite3;
c=sqlite3.connect('secrets_of_style.db'); r=c.execute('SELECT * FROM prendas');
w=csv.writer(open('prendas.csv','w',newline='',encoding='utf-8')); w.writerow([d[0]
for d in r.description]); w.writerows(r)"
```

Los respaldos contienen correos, resúmenes de contraseñas y, si existen, fotos de personas. Deben guardarse en una unidad con acceso restringido.

2.4 Lineamientos de seguridad de la base de datos

Los lineamientos siguen tres principios: cada persona solo ve sus datos, el servidor no confía en lo que manda la app y los datos sensibles nunca salen de la computadora. La Tabla 17 indica cómo se aplica cada uno.

Tabla 17. Lineamientos de seguridad aplicados

Lineamiento	Cómo se aplica	Dónde
Contraseñas que no se pueden leer	PBKDF2-HMAC-SHA256, 100 000 iteraciones y sal aleatoria de 16 bytes; se comparan en tiempo constante. Mínimo 8 caracteres.	seguridad.generar_hash
Sesiones que no se pueden falsificar	Token usuario.versión.vencimiento.firma con HMAC-SHA256; vence a los 7 días; cerrar sesión sube la versión y anula los tokens de todos los dispositivos.	seguridad.crear_token, usuario_actual
Clave fuera del código	Variable CLAVE_SECRETA o archivo .clave_secreta aleatorio, excluido de git.	seguridad.leer_clave_secreta; .gitignore
Aislamiento por cuenta	Toda consulta filtra por usuario_id; un recurso ajeno responde 404, sin revelar que existe.	buscar_prenda; buscar_prueba
Sin inyección SQL	Todas las consultas usan parámetros (?). Los nombres de columna de los filtros están fijos en el código.	principal.py; probador.py
Integridad de los datos	Llaves foráneas activas, borrado en cascada, valores únicos, NOT NULL y validación de catálogos y rangos con Pydantic antes de escribir.	base_datos.py; principal.py
No confiar en la app	Las características de un voto y el orden de las pasadas del probador se calculan en el servidor con los datos de la base.	guardar_voto; probador.pasadas_de
Archivos seguros	Se valida la firma de los bytes (no el nombre) y el tamaño máximo; los nombres son UUID, imposibles de adivinar.	principal.revisar_imagen
Datos sensibles privados	Las fotos del probador van a backend\privado, que no se publica; se entregan solo con token. La ruta nunca se envía a la app.	probador.py; leer_perfil

Lineamiento	Cómo se aplica	Dónde
Mínimo privilegio en servicios	El motor del probador solo escucha en 127.0.0.1 y solo lee o escribe dentro de privado y archivos_subidos.	motor.ruta_segura
IA sin terceros	Las fotos se analizan con Ollama y CatVTON en la misma computadora; solo las coordenadas aproximadas salen a Open-Meteo.	vision.py; motor
Base y secretos fuera del repositorio	*.db, archivos_subidos, privado y .clave_secreta están en .gitignore.	backend\.gitignore
Origen de las peticiones	CORS solo permite localhost y redes privadas.	principal.ORIGENES_PERMITIDOS

Riesgos que siguen abiertos. La Tabla 18 lista lo que no está resuelto y cómo se mitigaría. Son aceptables para un sistema que corre en una red local de confianza, pero no para publicarlo en internet.

Tabla 18. Riesgos residuales

Riesgo	Situación actual	Mitigación propuesta
Tráfico sin cifrar	La app y el servidor hablan por HTTP en la red local.	HTTPS con un proxy inverso y certificado; no usar redes Wi-Fi públicas.
Fotos de prendas públicas	/archivos se sirve sin sesión si se conoce el nombre UUID.	Servirlas con token, como ya se hace con el probador.
Base sin cifrar en disco	Quien tenga acceso al archivo puede leer correos y datos.	Cifrar el disco con BitLocker o usar SQLCipher.
Intentos de inicio de sesión ilimitados	Se pueden probar contraseñas sin límite.	Limitar intentos por IP y por cuenta.
Respaldos manuales	Si nadie los hace, una falla del disco borra todo.	Programar el respaldo semanal (sección 2.3).
Sin roles de administración	No hay cuenta de administrador; el mantenimiento se hace sobre el archivo.	Agregar un rol y registrar quién hace cada cambio.

3. Arquitectura de software

3.0 Requerimientos funcionales y no funcionales

Los requerimientos funcionales (RF) describen lo que hace el sistema; los no funcionales (RNF), cómo debe comportarse. Los RF-01 a RF-22 coinciden con el manual técnico; RF-23 y RF-24 corresponden al probador virtual.

Tabla 19. Requerimientos funcionales

Código	Requerimiento	Estado	Dónde se implementa
RF-01	Registrar una cuenta con correo y contraseña (mínimo 8 caracteres).	Implementado	POST /api/auth/registro; iniciar-sesion.jsx
RF-02	Iniciar y cerrar sesión; cerrar sesión invalida los tokens en todos los dispositivos.	Implementado	/api/auth/login, /cerrar-sesion; sesion.jsx
RF-03	Registrar una prenda con foto, categoría, subcategoría, color, abrigo, impermeabilidad, estampado y estilo.	Implementado	POST /api/prendas; nueva-prenda.jsx
RF-04	Listar prendas con filtros.	Implementado	GET /api/prendas; index.jsx
RF-05	Ver el detalle de una prenda y cambiar su estado.	Implementado	PATCH /api/prendas/{id}/estado; prenda/[id].jsx
RF-06	Eliminar una prenda, su foto y sus apariciones.	Implementado	DELETE /api/prendas/{id}
RF-07	Consultar el clima con caché de 30 minutos y respaldo sin conexión.	Implementado	GET /api/clima; clima.py
RF-08	Armar hasta tres atuendos a partir de una prenda base y una ocasión.	Implementado	POST /api/recomendaciones; busqueda.py
RF-09	Explicar cada atuendo con motivos, ventajas para el clima y avisos.	Implementado	logica.py; busqueda.py
RF-10	Guardar, listar y eliminar atuendos con el clima del día.	Implementado	/api/atuendos; guardados.jsx
RF-11	Mostrar estadísticas y gráficas del armario.	Implementado	GET /api/estadisticas; estadisticas.jsx
RF-12	Usar la ubicación del teléfono; si no hay permiso, usar Ciudad de México y avisarlo.	Implementado	generador.jsx
RF-13	Adaptarse al modo claro u oscuro y a "reducir movimiento".	Implementado	tema.js; movimiento.jsx
RF-14	Registrar altura y peso y calcular IMC y complexión.	Solo en la API	GET y PUT /api/perfil
RF-15	Detectar con IA los datos de una prenda y autocompletar el formulario.	Implementado	POST /api/prendas/analizar; vision.py
RF-16	Corregir estampado y estilo a mano o con IA.	Implementado	PATCH …/estilo; POST …/detectar-estilo
RF-17	Armar atuendos según la ocasión, que pesa más que color y clima.	Implementado	logica.py; busqueda.py
RF-18	Explicar un atuendo en lenguaje natural con un modelo local.	Implementado	POST /api/recomendaciones/explicacion

Código	Requerimiento	Estado	Dónde se implementa
RF-19	Aprender gustos con un corazón; sin corazón cuenta como "no me gusta".	Implementado	POST /api/atuendos/voto; Perchero.jsx
RF-20	Reordenar los candidatos de A* con el perceptrón sin pasar sobre las reglas.	Implementado	aprendizaje.reordenar
RF-21	Cuestionario de estilo inicial y opcional.	Implementado	PUT /api/preferencias; cuestionario.jsx
RF-22	Buscar prendas por nombre, categoría, color o estampado.	Implementado	index.jsx
RF-23	Probar un look en la foto de la persona: se viste la parte inferior y luego la superior o el abrigo.	Servidor listo; pantalla en desarrollo	/api/atuendos/probador; probador.py; motor
RF-24	Subir, cambiar y borrar la foto del probador, con todo lo generado con ella.	Servidor listo; pantalla en desarrollo	/api/perfil/foto

Tabla 20. Requerimientos no funcionales

Código	Categoría	Requerimiento y valor medido
RNF-01	Seguridad	Contraseñas con PBKDF2-HMAC-SHA256 (100 000 iteraciones); tokens firmados con HMAC-SHA256 que vencen a los 7 días.
RNF-02	Rendimiento	A* con 12 candidatos tarda unos 6 ms por prenda base en el armario de 36 prendas.
RNF-03	Usabilidad	Contraste de al menos 4.5:1 (WCAG AA) en ambos temas; áreas táctiles de 44 pt o más.
RNF-04	Compatibilidad	iOS y Android con Expo Go (SDK 58) y versión web en Chrome o Edge.
RNF-05	Privacidad	Las fotos se procesan en la misma computadora; solo la ubicación aproximada sale a Open-Meteo.
RNF-06	Mantenibilidad	Código en español separado por responsabilidades; 244 comprobaciones automáticas en 6 scripts.
RNF-07	Portabilidad	Servidor con la biblioteca estándar de Python más FastAPI, Uvicorn y python-multipart; base de datos en un archivo.
RNF-08	Disponibilidad	Requiere la computadora encendida y en la misma red Wi-Fi que el teléfono.
RNF-09	Rendimiento de la IA	Detectar una prenda con gemma3:4b toma unos 4 s por foto.
RNF-10	Tolerancia a fallos	Sin Ollama o sin el motor, la API responde 503 con un aviso y el resto de la app sigue funcionando.
RNF-11	Rendimiento del probador	En la RTX 4050: 28 a 38 s por prenda a 768×1024 con 20 pasos y 5.4 GB de VRAM de pico; un look con dos prendas, alrededor de un minuto. Las imágenes ya generadas se entregan al instante.
RNF-12	Uso de recursos	Gemma y CatVTON nunca ocupan la GPU al mismo tiempo; si el probador la tiene, la IA de texto avisa en 30 s en vez de bloquear la app.

3.1 Justificación de la arquitectura seleccionada

Se eligió una **arquitectura cliente-servidor en capas**, con un servidor modular (monolito modular) y dos **servicios de IA locales** separados del servidor: Ollama y el motor del probador. La app solo presenta e interactúa; el servidor concentra los datos, las reglas y la inteligencia.

La decisión responde a cinco requisitos concretos del proyecto:

1. **La IA necesita la GPU de la laptop.** Los modelos de visión, de lenguaje y de generación de imágenes no caben en un teléfono. Al ponerlos en el servidor, cualquier teléfono puede usarlos.
2. **Las fotos no deben salir de la computadora (RNF-05).** Con los modelos locales, las fotos nunca se envían a la nube.
3. **Restricción de tecnologías.** El proyecto solo admite Python en el servidor y React Native en la app. La separación en capas permite que cada lado use su tecnología sin mezclarlas.
4. **Un solo lugar para los datos.** El perceptrón aprende de los votos y A* necesita todo el armario. Si cada teléfono tuviera su copia, los gustos aprendidos se perderían al cambiar de teléfono.
5. **Tolerancia a fallos.** Ollama y el motor son procesos aparte. Si uno se apaga o se queda sin memoria, solo se pierde su función (503 con aviso) y el resto sigue funcionando (RNF-10).

El motor del probador tiene su propio Python 3.11 porque CatVTON exige `torch 2.1.2`, que no existe para el Python 3.14 del servidor. Separarlo también evita que una falla de memoria de la GPU tumbe la API.

Tabla 21. Alternativas evaluadas

Alternativa	Ventaja	Por qué se descartó
Todo en el teléfono, sin servidor	Funciona sin red.	El teléfono no puede correr los modelos de IA; los datos y los gustos quedarían en un solo teléfono.
Servidor en la nube (Firebase o un VPS)	Disponible desde cualquier lugar.	Las fotos saldrían a terceros, haría falta una GPU rentada y se agregarían tecnologías fuera de la restricción.
Microservicios	Cada parte escala por separado.	Demasiada complejidad para un equipo pequeño y un solo servidor; las reglas, A* y el perceptrón comparten datos en cada petición.
Cliente-servidor en capas con servicios de IA locales (elegida)	Simple, privada, aprovecha la GPU y aísla los modelos pesados.	Depende de que la laptop esté encendida y en la misma red (RNF-08).

3.2 Diagrama de arquitectura de software

La Figura 2 muestra las cuatro capas del servidor y la app, y los tres servicios con los que se comunica la capa de lógica. Cada capa solo habla con la de abajo: la app nunca toca la base de datos ni los modelos.

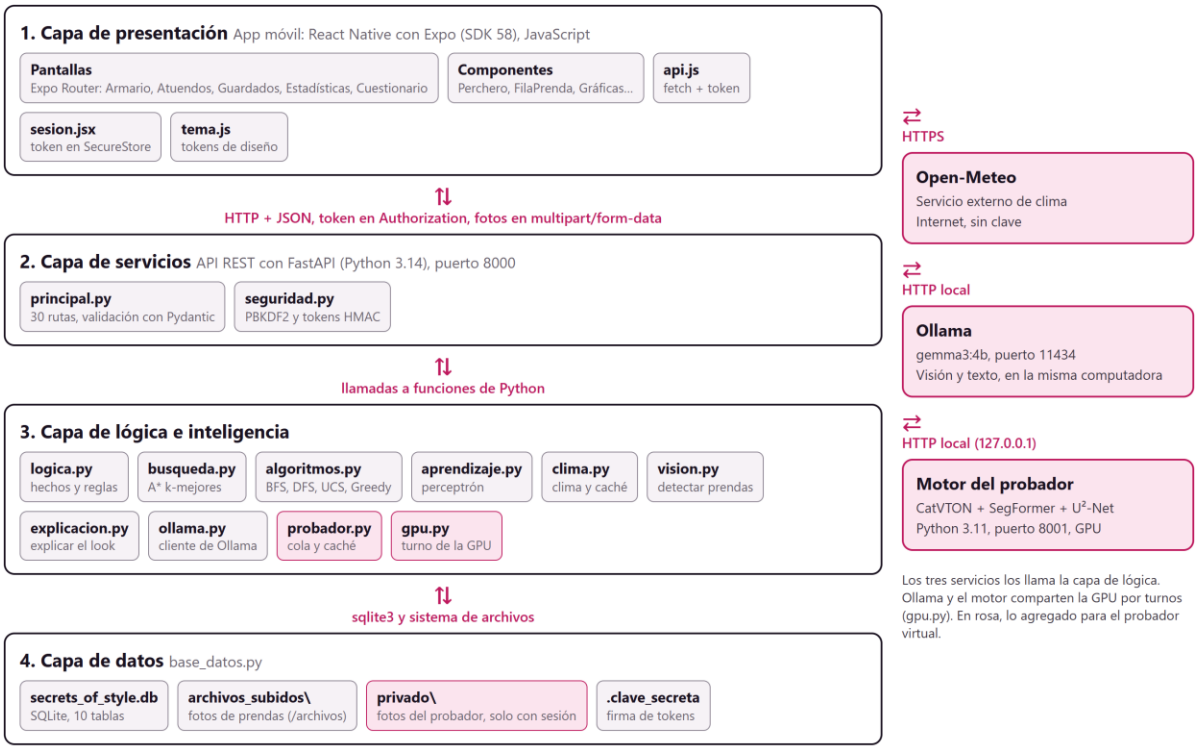


Figura 2. Arquitectura en capas de Secrets of Style y servicios de IA locales.

3.3 Características y funciones de los componentes

Tabla 22. Componentes del sistema

Componente	Capa	Función	Se comunica con
Pantallas (Expo Router)	Presentación	Armario, Atuendos, Guardados, Estadísticas, Cuestionario, Nueva prenda, Detalle de prenda e Inicio de sesión.	Componentes; api.js
Componentes de la app	Presentación	Piezas reutilizables: Perchero (look con corazón), FilaPrenda, GrupoPrendas, Gráficas, TarjetaClima, Selector, Boton, Campo.	Pantallas; tema.js
api.js	Presentación	Única puerta de la app al servidor: arma las peticiones, agrega el token y traduce los errores a mensajes.	API (HTTP)
sesion.jsx	Presentación	Guarda el token en SecureStore y decide si mostrar el inicio de sesión, el cuestionario o las pestañas.	api.js
tema.js y movimiento.jsx	Presentación	Tokens de color, tipografía y espacio en modo claro y oscuro; animaciones que respetan "reducir movimiento".	Pantallas
principal.py	Servicios	Define las 30 rutas, valida las entradas con Pydantic, aplica la sesión y coordina los módulos.	Todos los módulos
seguridad.py	Servicios	Resume contraseñas, firma y verifica tokens.	principal.py; SQLite
logica.py	Lógica	Proposiciones, reglas y encadenamiento hacia adelante; dice si una prenda es apta o está fuera de la ocasión.	busqueda.py; explicacion.py

Componente	Capa	Función	Se comunica con
busqueda.py	Lógica	A* k-mejores: arma los atuendos de menor costo y su accesorio.	logica.py; aprendizaje.py
algoritmos.py	Lógica	BFS, DFS, UCS, Greedy y heurísticas para comparar con A* (solo en el servidor).	busqueda.py
aprendizaje.py	Lógica	Perceptrón en Python puro: características, entrenamiento, pesos del cuestionario y reordenamiento.	principal.py
clima.py	Lógica	Consulta Open-Meteo, convierte el clima en hechos y lo guarda en caché.	Open-Meteo; SQLite
vision.py y explicacion.py	Lógica	Piden a Gemma detectar una prenda o redactar la explicación con los hechos de la lógica.	ollama.py
ollama.py	Lógica	Cliente HTTP de Ollama; espera el turno de la GPU y saca a CatVTON antes de usarla.	Ollama; gpu.py
probador.py	Lógica	Cola, pasadas, caché y archivos privados del probador; un hilo atiende una prueba a la vez.	Motor; SQLite; gpu.py
gpu.py	Lógica	Turno de la GPU y cliente del motor.	Motor; ollama.py; probador.py
base_datos.py	Datos	Conexión a SQLite, creación y migración del esquema, consultas con parámetros.	SQLite
SQLite y carpetas	Datos	secrets_of_style.db, archivos_subidos (fotos públicas) y privado (probador).	base_datos.py
Ollama (gemma3:4b)	Servicio local	Modelo de visión y lenguaje de 4 300 millones de parámetros, cuantizado a 4 bits.	ollama.py
Motor del probador	Servicio local	CatVTON viste la foto; SegFormer arma la máscara y U²-Net quita el fondo de la prenda.	gpu.py; probador.py
Open-Meteo	Servicio externo	Pronóstico del clima por coordenadas, sin clave.	clima.py

3.4 Diagramas de la arquitectura

Para una arquitectura cliente-servidor en capas, los diagramas que la explican son: el de **casos de uso**, que muestra qué hace cada actor; el de **despliegue**, que muestra en qué equipo corre cada componente; y los de **secuencia**, que muestran cómo cruzan las capas las peticiones más importantes. El modelo de datos está en la Figura 1 y la vista de capas en la Figura 2.

Diagrama de casos de uso

La persona usuaria es el actor principal. Ollama, Open-Meteo y el motor del probador son actores secundarios: sistemas que participan en algunos casos de uso. En rosa aparece el probador, que está en desarrollo.

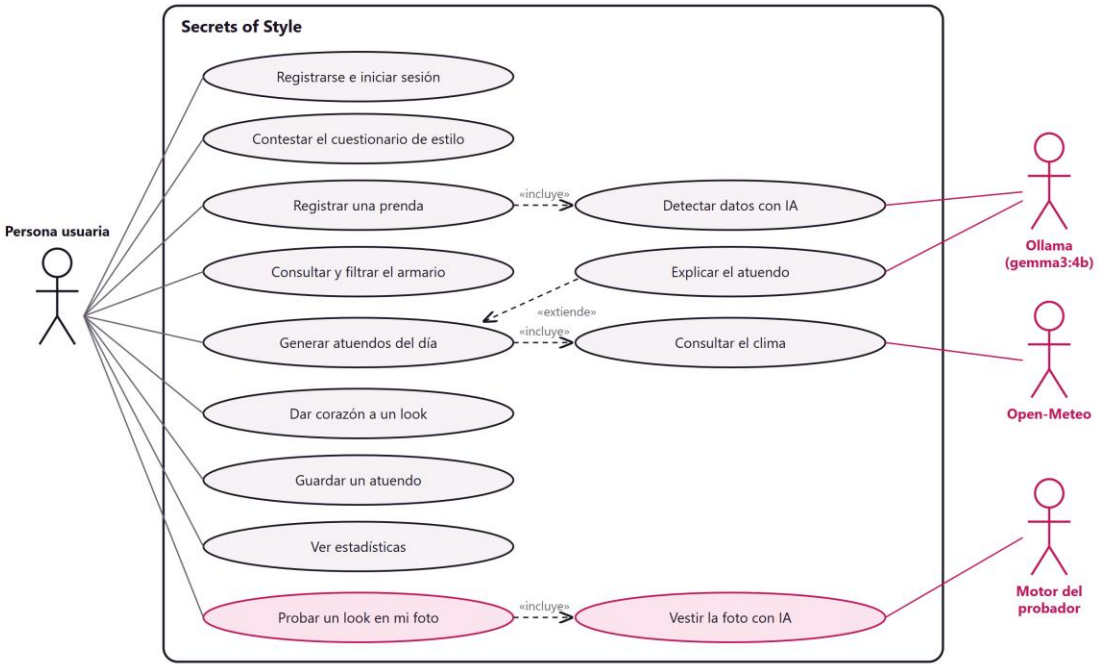


Figura 3. Casos de uso de Secrets of Style.

Diagrama de despliegue

Todo corre en la laptop del equipo, salvo la app, que corre en el teléfono dentro de Expo Go, y los dos servicios de internet. El motor del probador solo acepta conexiones de la misma computadora (127.0.0.1).

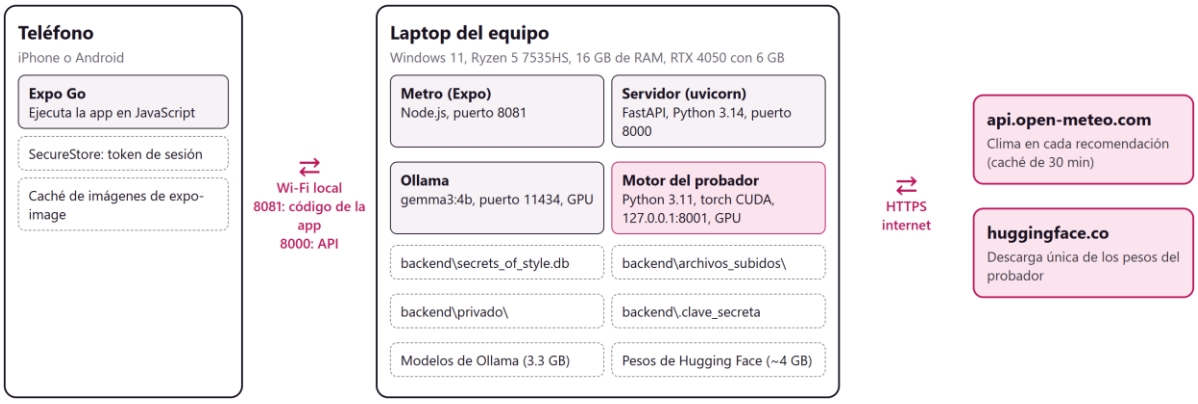


Figura 4. Despliegue: teléfono, laptop y servicios de internet.

Diagrama de secuencia: generar atuendos

Es la petición central del sistema. Pasa por las cuatro capas y combina las tres técnicas de IA del proyecto: lógica, búsqueda y aprendizaje.

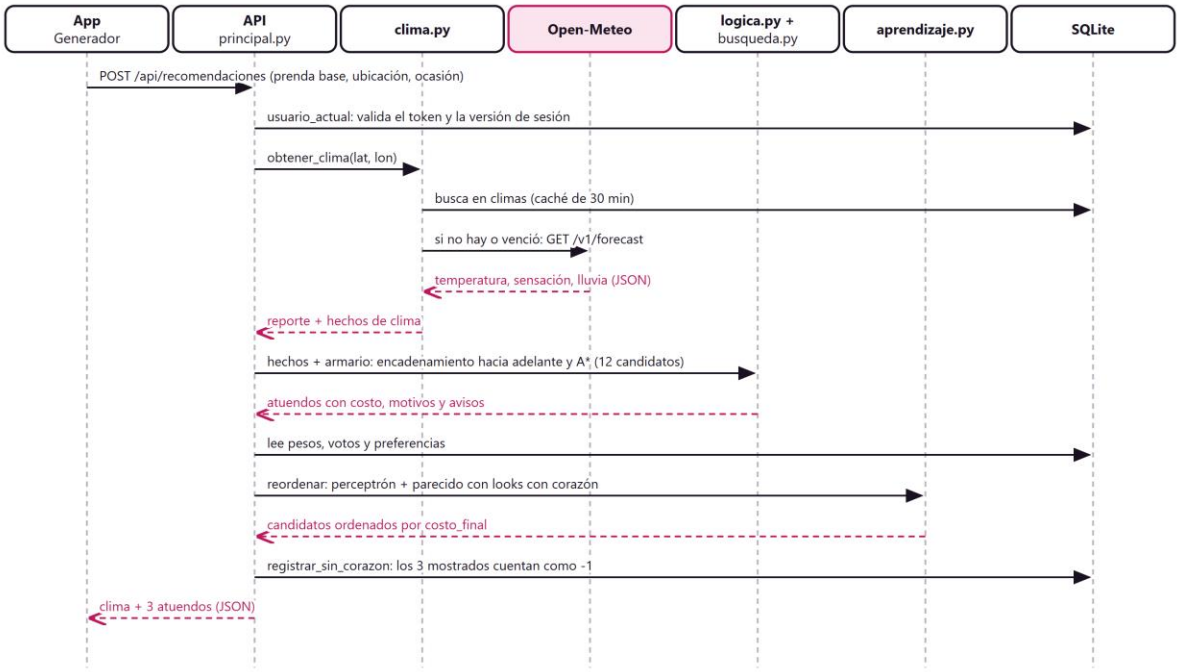


Figura 5. Secuencia de POST /api/recomendaciones.

Diagrama de secuencia: probar un look

El probador es asíncrono: la app recibe de inmediato el estado «en_cola» y consulta cada pocos segundos mientras un hilo del servidor genera la imagen con el motor. Así la app no se queda esperando más de un minuto con la conexión abierta.

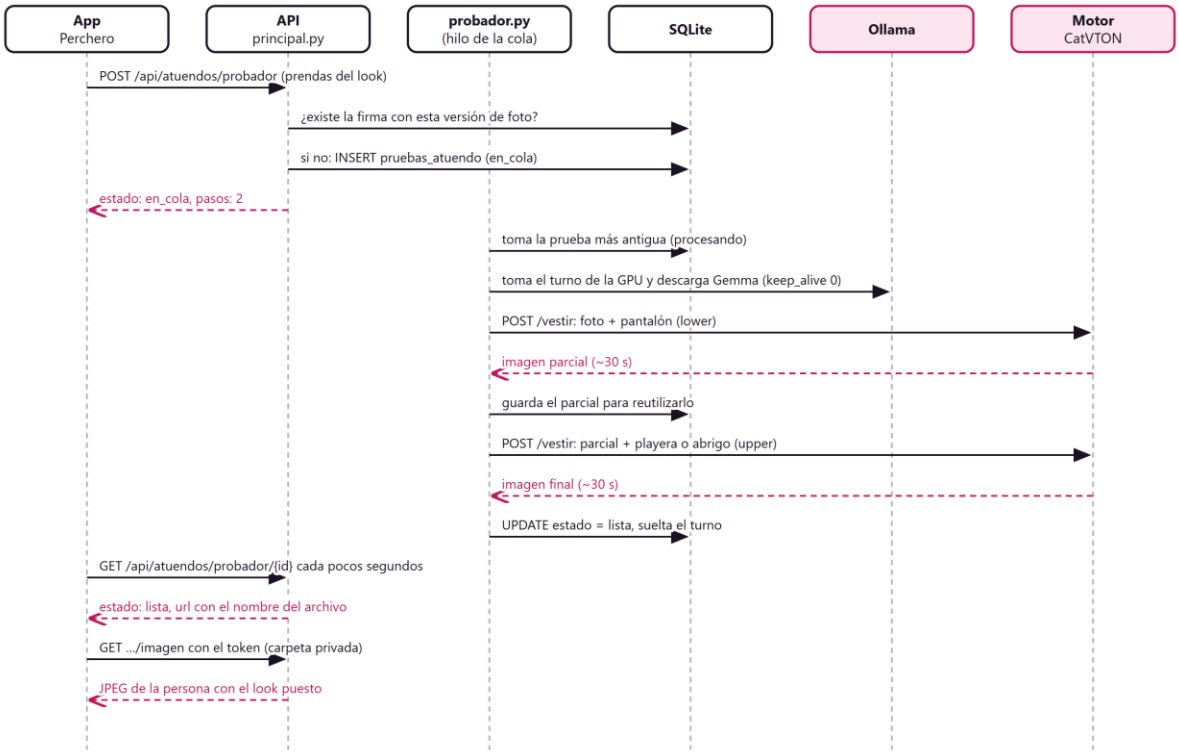


Figura 6. Secuencia del probador virtual (servidor listo; pantalla en desarrollo).